

OOP

programare orientată pe obiecte

CURS04: STATIC, FINAL, SINGLETON

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
Universitatea Politehnica Bucureşti

STATIC

STATIC VS INSTANTE

static desemneaza campuri si metode independente de instantele clasei

```
public class BasketBall {
  public static int size = 12;
  private String color ;

  Ball ( String color ) {
    this.color = color ;
  }
}
```

STATIC VS NON-STATIC CONTEXT

ceea ce se petrece într-o functie static ar trebui sa poata functiona chiar daca obiectul a fost instantiat sau nu

```
public class TestApp {
  int count ;
  public static void main ( String [] args ) {
    count ++;
  }
  // Cannot make a static reference to the non-static field coun
}
```

FINAL

FINAL PENTRU DATE PRIMITIVE (CONSTANTE)

```
public class Album {
  public final int tracks = 15;
  public final int releaseYear ;

  Album (int releaseYear ) {
    this.releaseYear = releaseYear ;
  }
}
```

FINAL PENTRU OBIECTE (CONSTANTE)

```
public class Boxer {
  public final int birthYear ;
  public String name ;

  Boxer ( String name , int birthYear ) {
    this.name = name ;
    this.birthYear = birthYear ;
  }
}
...
final Boxer champion = new Boxer ("Cassius Clay", 1942);
champion = new Boxer ("Mike Tyson", 1966);     // not ok
champion.name = " Muhammad Ali ";              // ok
```

FINAL PENTRU CAMPURI (IMUTABILITATE)

Daca toate atributele admit o unica initializare:

```
public class Ball {
  public final String color ;
  public Ball ( String color ) {
    this.color = color ;
  }
}
Ball basketball = new Ball (" brown ") ;
```

Asa sunt, spre exemplu,
obiectele de tip String sau Integer.

FINAL PENTRU CLASE (NU POT FI MOSTENITE)

Puteti folosi final pentru clase care doriti sa nu poata fi mostenite

```
public final class Animal {
...
}
public class Dog extends Animal {
...
}
// error: cannot inherit from nal Animal
```

IMMUTABLE VS COMPOSITION

Puteti marca drept final obiectele folosite in relatia de Compunere

```
public class Punct2D {
   ...
}
public class Circle {
  public final Punct2D centru ;
   ...
}
```

SINGLETON

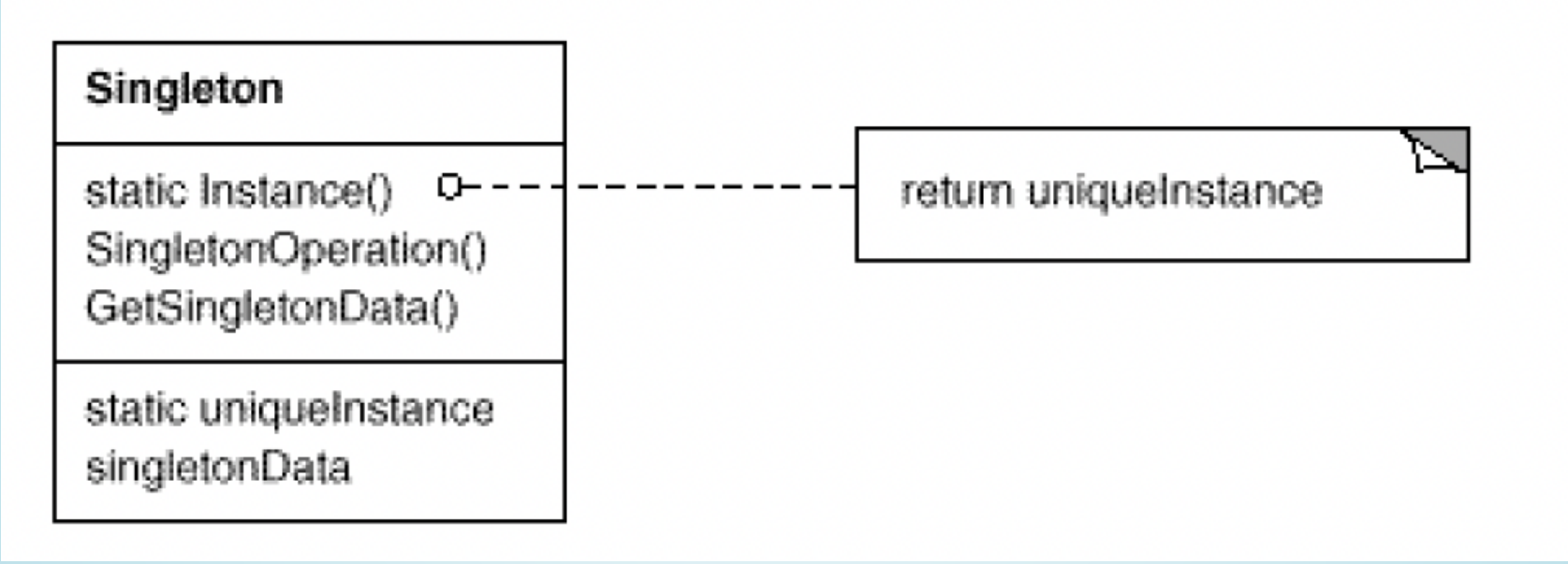
DESIGN PATTERNS

Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

-- Christopher Alexander

SINGLETON

How do we ensure that a class has only one instance and that the instance is easily accessible?



-- Gang of Four, Design Patterns